City Network & Job Scheduler

Design and Analysis of Algorithms · Assignment 3

Adilzhan Kuandykov | SE-2526 | 23 October 2026

Repository: https://github.com/adilzhandev/AITU-DAA-A3 | Release: v1.0

1. Implementation and experimental design

One adjacency-list Graph supports directed and undirected weighted multigraphs, including loops, parallel edges, isolated vertices and disconnected components. DSU uses union by size and iterative path compression. Kruskal and lazy-heap Prim return minimum spanning forests; Kosaraju uses an explicit stack; Kahn detects cycles; DAG shortest paths start at every original zero-indegree vertex and retain parents.

The exact Random(42) generator is restarted for each graph: n = 1,000, 3,162, 10,000, 31,623, 100,000; m = 4n or 20n. Each size/density supplies an undirected graph, an unrestricted directed graph and a DAG. The 50 CSV rows contain five runs per algorithm/graph, with median compute time. Generation and validation are outside the timer; there is no separate warm-up phase. Every run's result and counters are checked for determinism, both MST results agree, all DAG orders are validated and all supplied reference values pass.

Complexity of the submitted implementations

Operation	Time	Extra space	Reason
DSU find / union	O(α(V)) amortized	O(1) per call; O(V) structure	Size balancing + compression; makeSet costs O(V).
Kruskal	O(V + E log(E+1) + Eα(V))	O(V+E)	Sort a copied edge list, then DSU checks; stop at V-1 selected edges.
Prim (lazy heap)	O(V + E log(E+1))	O(V+E)	Scan adjacency lists; heap stores up to O(E) candidate edges.
Kosaraju	O(V+E)	O(V+E)	Two iterative DFS passes and an explicit transposed graph.
Kahn	O(V+E)	O(V)	In-degrees and queue; each vertex leaves once and each arc is removed once.
DAG shortest paths	O(V+E)	O(V)	Topological ordering plus one relaxation pass; distance and parent arrays.

V=n and E=m count original edges; an undirected edge occupies two adjacency entries. Space excludes the input graph and includes returned arrays/edges. For simple connected graphs log E = O(log V), but the implemented lazy Prim supports parallel edges, so O(E log E) is the precise heap bound. These linear-density inputs are both E=Θ(V); 'dense' here means 20n, not Θ(V²).

What the counters measure

dsuOps counts find calls plus successful unions, not individual parent-pointer hops. comparisons counts actual weight-comparator calls in sorting or the heap. edgeScans counts the main phase: Kruskal candidates, Prim adjacency visits, two Kosaraju DFS passes, Kahn outgoing-edge removals, or DAG relaxations. Transposition, in-degree initialization and the DAG algorithm's prerequisite Kahn pass are deliberately excluded from this phase counter. maxDepth is peak simultaneous explicit-stack entries, not an additive work count.

2. Correctness and verification

Cut property. Let A be an acyclic partial solution that is contained in some minimum spanning tree T. Consider a cut that no edge of A crosses, and let e be a lightest edge across that cut. If e is already in T, adding it to A is safe. Otherwise, adding e to T creates a cycle that contains another crossing edge f. Since w(e) ≤ w(f), replacing f by e preserves a spanning tree and cannot increase its weight. Kruskal applies this argument between DSU components, while Prim applies it between the visited vertices and the rest of the current connected component. Repeating the safe choice gives an MST, or independently one minimum tree per component for a disconnected graph.

Topological induction. Give every original zero-indegree vertex distance zero, equivalently adding a virtual source with zero-weight arcs to those vertices. Every vertex of a finite DAG is reachable from such a source. Assume that all vertices before u in a topological order already have their correct distances. Every incoming edge of u comes from one of those earlier vertices, so all possible final edges of a path to u have already been relaxed. The stored minimum is therefore the true shortest distance to u. Relaxing all outgoing edges of u maintains the induction, and parent updates preserve a path realizing each finite distance. The proof uses edge direction in the order rather than nonnegative weights, so negative edges are safe in a DAG, whereas general graphs need another method such as Bellman-Ford and reachable negative cycles prevent finite shortest-walk distances.

Verification: 81 JUnit tests pass with no failures, errors or skips. Coverage includes 1,000 random DSU unions against a label reference, 100 connected MST cross-checks, 80 exhaustive small forest oracles, 100 SCC graphs with up to 200 vertices against BFS reachability, 100 random DAGs against Bellman-Ford including negative weights and restored paths, cycle rejection and file parsing. The 100,000-vertex SCC path finishes with maxDepth=100,000. Three data/ fixtures exercise actual file loading, disconnected forests, SCC cycles, parallel edges, self-loops, isolated vertices and negative DAG weights.

Exact reference results reproduced

n	m	MST weight (both)	SCCs	Sum of DAG distances
1,000	4,000	147,970	27	396,274
1,000	20,000	30,363	1	252,411
3,162	12,648	468,911	137	1,271,911
3,162	63,240	97,589	1	723,157
10,000	40,000	1,486,139	373	3,941,828
10,000	200,000	309,550	1	2,381,342
31,623	126,492	4,715,515	1188	12,551,891
31,623	632,460	968,517	1	7,376,612
100,000	400,000	15,012,094	3923	40,096,539
100,000	2,000,000	3,056,681	1	23,512,087

Kahn returns n vertices in all ten cases. Reference results are from Assignment 3, section 7; all 50 rows and per-configuration checks are retained in results/results.csv and results/benchmark.log.

3. Operations and normalized complexity

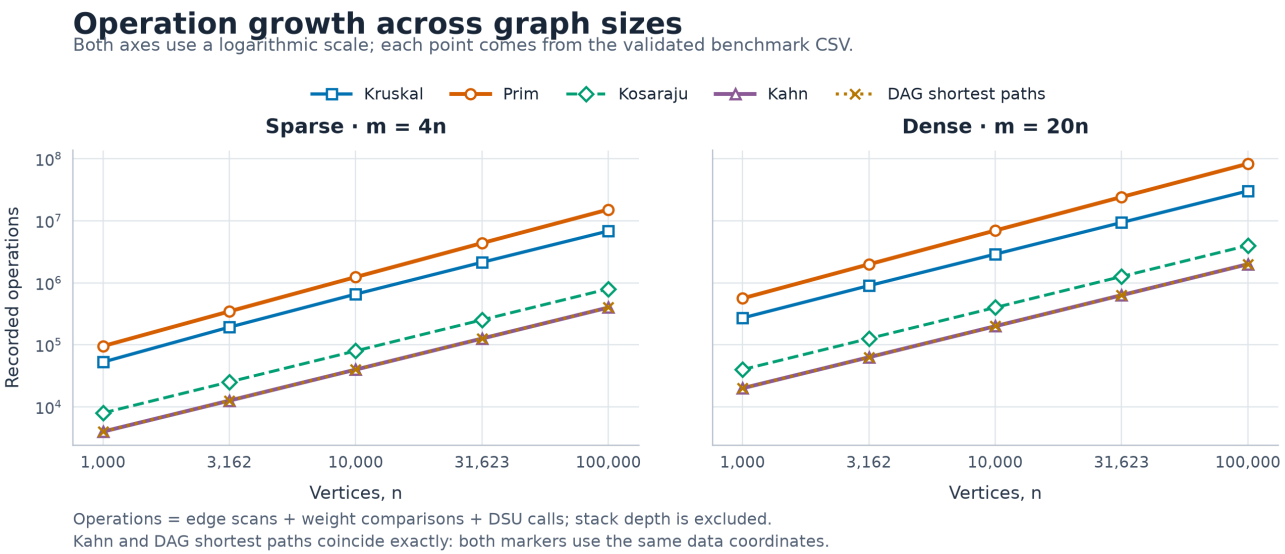


Figure 1. Selected operation total = edgeScans + comparisons + dsuOps; maxDepth is excluded. Both axes are logarithmic. Kahn and DAG paths overlap exactly.

The total combines distinct recorded events as a work proxy; it is neither an instruction count nor a prediction of elapsed time. MST work is dominated by weight comparisons, whereas traversal totals here consist only of counted edge visits. Ratios below use all five sizes independently within each density; a narrow range is evidence consistent with a formula, not a proof of an asymptotic bound.

Algorithm	Normalized counter	Sparse min to max	Dense min to max
Kruskal	comparisons / (m log₂m)	0.745 to 0.907	0.666 to 0.903
Kruskal	dsuOps / m	1.791 to 2.248	0.465 to 0.794
Prim	comparisons / (m log₂m)	1.829 to 1.934	1.825 to 1.890
Prim	edgeScans / (n+m)	1.600 to 1.600	1.905 to 1.905
Kosaraju	edgeScans / (n+m)	1.600 to 1.600	1.905 to 1.905
Kahn	edgeScans / (n+m)	0.800 to 0.800	0.952 to 0.952
DAG paths	edgeScans / (n+m)	0.800 to 0.800	0.952 to 0.952

Traversal ratios are exactly flat because m=cn: 2m/(n+m)=2c/(1+c) and m/(n+m)=c/(1+c). Prim's heap comparisons grow at the expected near m log m scale; Kruskal's comparison ratio is not exactly constant, partly because only 1,000 distinct weights occur and sorting adapts to the input. Its DSU ratio also varies as early stopping examines a changing fraction of edges. For k examined candidates and C components, dsuOps = 2k + n - C; thus dsuOps/k remains bounded between 2 and 3. Counting calls cannot empirically establish α(n): that bound comes from union-by-size plus path compression.

4. Edge visits and Kruskal's early stop

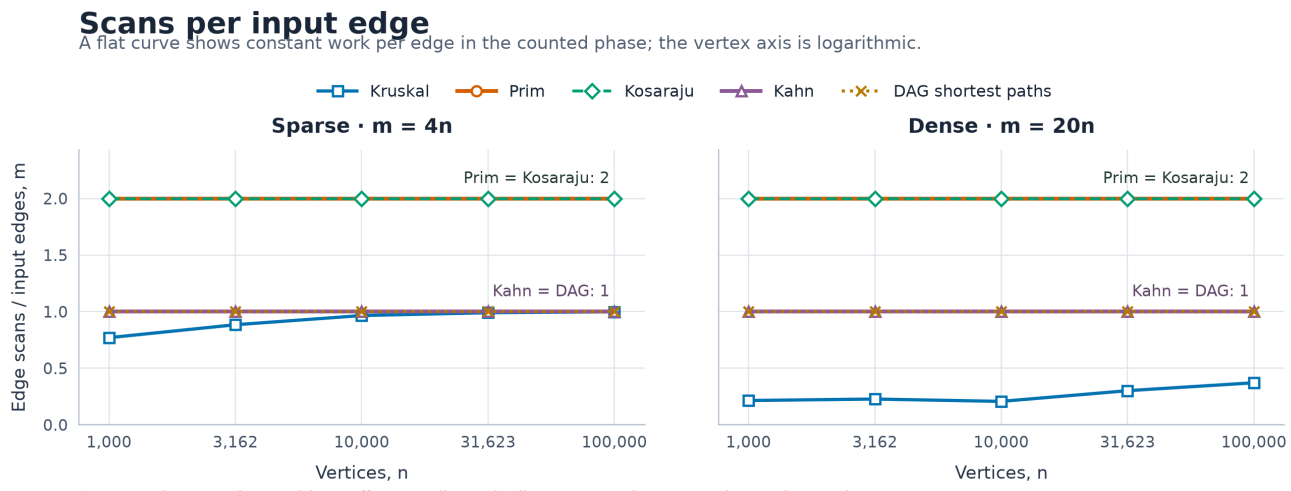


Figure 2. Original edges m are the denominator. Prim and Kosaraju are exactly 2; Kahn and DAG relaxation are exactly 1. Coincident curves are labelled.

Prim's factor two comes from the undirected adjacency representation; Kosaraju's factor two comes from its two directed DFS passes. Their equal counts do not mean equal tasks, memory usage or elapsed time. Kahn and DAG paths also have different preprocessing although their reported phase scans coincide.

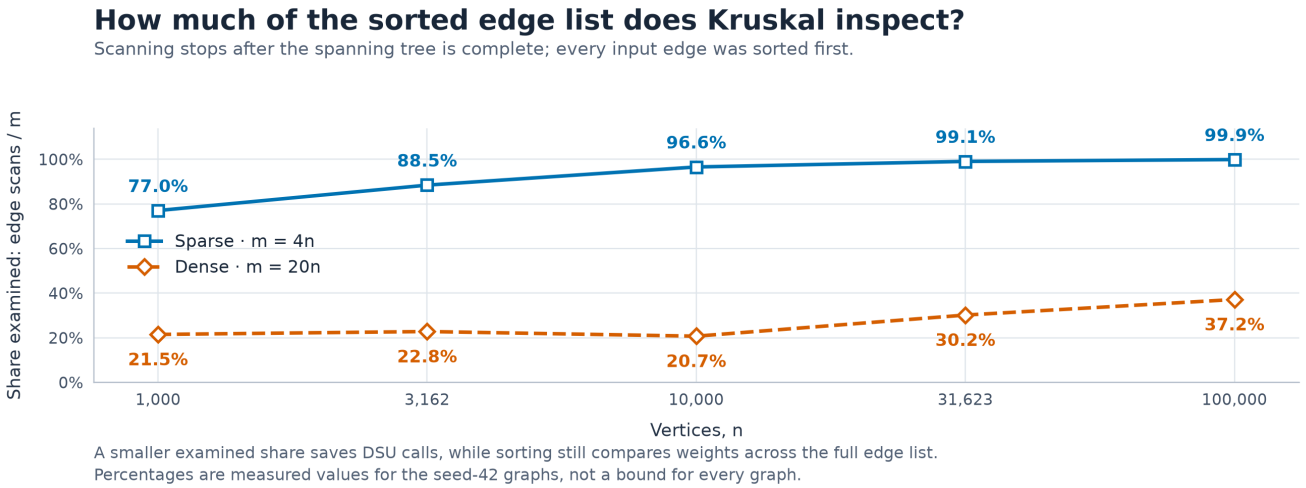


Figure 3. Kruskal stops its DSU scan after choosing n-1 edges, but first sorts all m input edges.

The sparse examined share rises from 77.0% to 99.9%; with fewer alternative connections it approaches the whole edge list. Dense shares range from 20.7% to 37.2%, so many expensive candidates never reach DSU. This saves scanning and union/find calls, not sorting. Prim still visits all 2m adjacency entries and manages its lazy heap. The measured comparison counts favor Kruskal on these generated inputs; these curves alone do not imply a universal winner.

5. Slopes, algorithm choice and timing

Curve	Sparse slope	Dense slope	Minimum R^2
Prim scans	1.000	1.000	1.0000
Kosaraju scans	1.000	1.000	1.0000
Kahn / DAG scans	1.000	1.000	1.0000
Kruskal scans	1.055	1.119	0.9961
Kruskal sort comparisons	1.055	1.016	0.9995
Prim heap comparisons	1.108	1.090	0.9999

OLS fits log(counter) = a + b log(n) on the five measured sizes. Traversal slopes are exactly 1.000. Kruskal's scans are slightly superlinear here because its examined fraction changes, not because edgeScans includes sorting. The E log E term belongs to the comparison phase; the assignment's slope wording conflates these counters. Finite slopes and R^2 describe this seed and size range, not a proof or a general performance ranking.

Situation	Choice and reason
Sparse undirected graph; edge list	Kruskal: direct sort plus DSU, straightforward forest handling.
Undirected graph; adjacency list	Prim grows a component naturally. This lazy heap uses O(E) space; compare counts before preferring it.
Truly dense simple graph, $E \approx V^2$	An $O(V^2)$ array/matrix Prim variant can avoid heap overhead; that variant is not implemented here.
Directed graph may have cycles	Kosaraju groups mutual reachability/deadlocks; Kahn detects cycles when only scheduling validity is needed.
Acyclic jobs / weighted pipeline	Kahn for a schedule; topological relaxation for paths, including negative weights.
General weighted directed graph	DAG relaxation is invalid if cyclic. Dijkstra needs nonnegative weights; Bellman-Ford supports negative edges and detects reachable negative cycles.

Timing. On the largest dense graph, median times in ms were Kruskal 543.450, Prim 2,388.564, Kosaraju 716.896, Kahn 162.421, DagShortestPath 217.691. Environment: openjdk version "17.0.20.1" 2026-08-18; Ubuntu 26.04.1 LTS; Intel(R) N100; -Xms256m -Xmx3g. Five-run medians reduce outliers but do not remove JVM compilation, allocation/GC, background load or hardware effects; the small inputs are especially noisy. Counters and reference checks are the primary evidence. Machine details and test totals are recorded in results/environment.json.

Reproduction and provenance. After installing scripts/requirements.txt, run bash scripts/reproduce.sh to test, benchmark and rebuild the three plots plus this report. The Java tests and benchmark are the correctness evidence; scripts/plot_results.py validates the CSV and stores numerical fits in results/analysis.json. Main, five feature branches and tag v1.0 identify the submitted release. Source: Taubakabyl Nurlybek, DAA Assignment 3 - Graphs (supplied specification), sections 1-10. AI acknowledgement: the student supplied the initial Java implementations; OpenAI Codex assisted with auditing, additional tests, benchmark timing isolation, plots, report and release preparation.